

ASSIGNMENT 10

1. Write a procedure that computes the perimeter and the area of a rectangle. Define your own values for the length and width. (Assuming that L and W are the length and width of the rectangle, Perimeter = $2*(L+W)$ and Area = $L*W$.)

```
root DELIMITER $$
root
root CREATE PROCEDURE pa_measures(l INT, w INT)
  -> BEGIN
  ->   DECLARE v_perimeter INT;
  ->   DECLARE v_area INT;
  ->
  ->   SET v_perimeter = 2 * (l + w);
  ->   SET v_area = l * w;
  ->
  ->   SELECT v_perimeter AS por, v_area AS aor;
  -> END
  -> $$
Query OK, 0 rows affected (0.02 sec)

root
root DELIMITER ;
root call pa_measures(7,4);
+-----+-----+
| por  | aor  |
+-----+-----+
|  22  |  28  |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

2. Write a procedure that declares an integer variable called num, assigns a value to it, computes and inserts into the temp table the value of the variable itself, its square, and its cube.

```

root DELIMITER $$
root
root CREATE PROCEDURE sp_num_powers()
-> BEGIN
->     DECLARE num INT;
->
->     -- Assign a value to the variable
->     SET num = 5;
->
->     -- Create the temp table (drops any leftover one first, s
since TEMPORARY tables
->     -- persist for the session and re-calling the procedure w
ould otherwise error)
->     DROP TEMPORARY TABLE IF EXISTS temp_powers;
->     CREATE TEMPORARY TABLE temp_powers (
->         num_value INT,
->         num_square INT,
->         num_cube INT
->     );
->
->     -- Insert the value, its square, and its cube
->     INSERT INTO temp_powers (num_value, num_square, num_cube)
-
->     VALUES (num, num * num, num * num * num);
->
->     -- Show the result
->     SELECT * FROM temp_powers;
-> END
-> $$
Query OK, 0 rows affected (0.01 sec)

```

```

root
root DELIMITER ;
root call sp_num_powers();
+-----+-----+-----+
| num_value | num_square | num_cube |
+-----+-----+-----+
|          5 |          25 |         125 |
+-----+-----+-----+
1 row in set (0.04 sec)

Query OK, 0 rows affected (0.04 sec)

```

3. Create a procedure to Convert a temperature in Fahrenheit (F) to its equivalent in Celsius (C) and vice versa. The required formulae are

$$C = (F - 32) * 5/9 \quad F = 9/5 * C + 32$$

```

root DELIMITER $$
root
root CREATE PROCEDURE con(F DOUBLE, C DOUBLE)
-> BEGIN
->     DECLARE v_fahrenheit DOUBLE;
->     DECLARE v_celsius DOUBLE;
->
->     SET v_celsius = (F - 32) * 5/9;      -- F converted TO celsius
->     SET v_fahrenheit = (9/5) * C + 32;  -- C converted TO fahrenheit
->
->     SELECT F AS input_f, v_celsius AS f_converted_to_c,
->           C AS input_c, v_fahrenheit AS c_converted_to_f;
-> END
-> $$
Query OK, 0 rows affected (0.02 sec)

root
root DELIMITER ;
root CALL con(98.6, 100);
+-----+-----+-----+-----+
| input_f | f_converted_to_c | input_c | c_converted_to_f |
+-----+-----+-----+-----+
|    98.6 |             37 |    100 |             212 |
+-----+-----+-----+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

```

4. Create a procedure to Convert a number of inches into yards, feet, and inches. For example, 124 inches equals 3 yards, 1 foot, and 4 inches.

```

root  DELIMITER $$
root  CREATE PROCEDURE noi() ^C
root  DELIMITER $$
root
root  CREATE PROCEDURE inches_convert(IN total_inches INT)
-> BEGIN
->     DECLARE v_yards INT;
->     DECLARE v_feet INT;
->     DECLARE v_inches INT;
->     DECLARE v_remainder INT;
->
->     -- 1 yard = 36 inches
->     SET v_yards = total_inches DIV 36;
->     SET v_remainder = total_inches MOD 36;
->
->     -- 1 foot = 12 inches
->     SET v_feet = v_remainder DIV 12;
->     SET v_inches = v_remainder MOD 12;
->
->     SELECT total_inches AS input_inches,
->            v_yards AS yards,
->            v_feet AS feet,
->            v_inches AS inches;
-> END
-> $$

```

Query OK, 0 rows affected (0.03 sec)

```

root
root  DELIMITER ;
root  CALL inches_convert(124);
+-----+-----+-----+-----+
| input_inches | yards | feet | inches |
+-----+-----+-----+-----+
|          124 |      3 |      1 |        4 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

Query OK, 0 rows affected (0.00 sec)

5. Create a procedure to read two real numbers and tell whether the product of the two numbers is equal to or greater than 100.

```
root DELIMITER $$
root
root CREATE PROCEDURE compare_product(IN num1 DOUBLE, IN num2 DOUBLE)
-> BEGIN
->     DECLARE v_product DOUBLE;
->     DECLARE v_result VARCHAR(30);
->
->     SET v_product = num1 * num2;
->
->     IF v_product >= 100 THEN
->         SET v_result = 'Equal to or greater than 100';
->     ELSE
->         SET v_result = 'Less than 100';
->     END IF;
->
->     SELECT num1 AS number1,
->            num2 AS number2,
->            v_product AS product,
->            v_result AS comparison_result;
-> END
-> $$
```

Query OK, 0 rows affected (0.02 sec)

```
root
root DELIMITER ;
root CALL compare_product(12.5, 9);
```

number1	number2	product	comparison_result
12.5	9	112.5	Equal to or greater than 100

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)